Converting an SS3 model to opal

A step-by-step guide, worked through with the IO yellowfin SS3 model

Nicholas Ducharme-Barth

2026-10-15

Table of contents

1 Purpose 1
1.1 How the converter works 2
2 What you need 3
3 The conversion settings 3
4 Read the SS3 model 6
5 Convert 7
6 Test the conversion 8
6.1 Test 1: structure 9
6.2 Test 2: reconciliation with SS3 10
6.3 Test 3: build the opal object 11
6.4 Test 4: bridging at SS3′s parameters 11
6.5 Starting objective and gradient 13
7 Set up and fit the final model 13
7.1 Seed opal's selectivity from SS3′s 13
7.2 Choose what to estimate 14
7.3 Fit 15
7.4 Save 16
8 A first look at the fit 16
9 From yellowfin to your model 20
10 Credit 22
11 Session 22

1 Purpose

This guide shows how a Stock Synthesis 3 (SS3) assessment is converted into an opal model, how to test that the conversion is right, and how to set up and fit the opal model. It works through one example from start to finish: the IATTC team's SS3 model of the Indian Ocean yellowfin (IO YFT) simulated data, which became opal fit **C2**. It is written for analysts converting their own SS3 model, and section 9 lists what to compare between your model and this one.

All the code is in one companion script, `code/rtmb/scripts/05-io-yft-ss3-to-opal-workflow.r`. The code shown below is taken from that script, not retyped. Copy the script and edit it for your model. The results here are from the script's own run, read from the cache it writes.

This guide covers the workflow only. For how the opal fits compare with SS3 and with the simulated truth, and why they differ, see:

- `ss3-to-opal-io-yft.qmd`: the first conversion, with both biology routes, diagnostic fits and the comparison with the operating model.
- `ss3-to-opal-io-yft-age-k.qmd`: SS3′s age-specific K growth in opal, and the selectivity fits C and C2.

- `code/rtmb/sims/sims-synthesis.qmd`: simulations of the catch-timing and selectivity differences between the two platforms.

1.1 How the converter works

The converter is a set of R functions in `code/rtmb/helper-fns/`:

File	What it does
`ss3-read.R`	Reads the SS3 files with r4ss and records their checksums
`ss3-convert.R`	Converts dimensions, fleets, catch, indices, length compositions, selectivity and parameters (`ss3_to_opal()`)
`ss3-biology.R`	Converts biology and growth: M, weight, maturity, spawning potential, length-at-age
`ss3-validate.R`	Tests: structural checks, reconciliation with SS3, the bridging replay
`ss3-opal-obj.R`	Builds the opal object and switches selectivity between SS3's and opal's
`ss3-fit-tools.R`	Newton polishing after `nlminb`, and extraction helpers

The converter follows three rules:

1. **Derived quantities come from `Report.sso`** wherever SS3 reports them: M, weight, maturity and spawning potential at age, selectivity at age, unfished spawning biomass and the recruitment bias ramp. These quantities already include every transformation SS3 applied internally, so nothing has to be re-derived from the control file.
2. **Every decision that cannot be read from the SS3 files is in one settings list.** That includes how to split an index, which fleets' compositions to use and which opal curve to use for each fleet. The functions hard-code nothing about the model.
3. **Where opal cannot represent SS3 exactly, the converter reports how close it gets** rather than hiding the approximation.

The workflow has eight steps:

Step	Section	Main functions	Output
1. Settings	3	an R list	`cfg`
2. Read SS3	4	`ss3_read_model()`	`ss3`
3. Convert	5	`ss3_to_opal()`, `prep_lf_data()`	`converted`
4. Test inputs	6.1–6.2	`validate_opal_inputs()`, `ss3_reconcile()`	check tables
5. Build	6.3	`ss3_opal_obj()`	`assessment`
6. Test dynamics	6.4–6.5	`ss3_use_external_selectivity()`, `ss3_bridge_table()`	bridging table
7. Set up and fit	7	`ss3_seed_par_sel()`, `ss3_fit_use_estimated_selectivity()`, `opal_fit()`	

Step	Section	Main functions	Output
8. Inspect and save	7–8	`opal_report()`, `opal_save()`	results, saved fit

2 What you need

- R with **opal** ≥ 0.1.1 (`dev` branch; 0.1.1 adds SS3's age-specific K growth), **r4ss**, **data.table**, **magrittr** and **this.path**.
- The SS3 model folder, with these files from a converged run:
  - `data.ss_new` (data);
  - `control.ss_new` (control);
  - `ss3.par` (parameter estimates; called `ss.par` in some versions);
  - `Report.sso`, which the converter needs for biology, selectivity and B0.
- The converter functions (`code/rtmb/helper-fns/`).

```
library(data.table)
library(magrittr)
library(opal)
library(r4ss)

stopifnot("opal >= 0.1.1 is required for SS3 age-specific K growth" =
              utils::packageVersion("opal") >= "0.1.1")
```

```
proj_dir = this.path::this.proj()
dir_fns  = file.path(proj_dir, "code", "rtmb", "helper-fns")
dir_out  = file.path(proj_dir, "code", "rtmb", "vignettes", "data")
dir.create(dir_out, recursive = TRUE, showWarnings = FALSE)

# The converter: ss3-read.R, ss3-convert.R, ss3-biology.R, ss3-validate.R,
# ss3-opal-obj.R and ss3-fit-tools.R.
for (f in list.files(dir_fns, pattern = "\\.[Rr]$", full.names = TRUE)) source(f)
```

3 The conversion settings

Everything the converter needs to know beyond the SS3 files is in `cfg`, a plain R list. The table below lists each block. The values below are for IO YFT, and each needs to be checked again for another model.

Block	Decision	IO YFT choice
`time`	Which SS3 time steps become opal time steps	All 256; each SS3 "year" is a quarter, so opal has `n_year = 256`, `n_season = 1`
`age`	Which SS3 ages to keep	1–28, so the opal age equals the SS3 age (age 0 dropped)
`growth`	How opal computes length-at-age	`ss3_age_k`: SS3's `GrowthModel 3` reproduced exactly
`bio`	Where M, weight, maturity and spawning potential come from	`endgrowth`: Report.sso's age table, used directly
`index`	How SS3's index fleets map to opal index series	Fleet 17 split into four quarterly series (opal 17–20)

Block	Decision	IO YFT choice
`lf`	Which fleets' length compositions to use, and the bin grid	Fleets 1–14 and 16; last bin regularised to 195–200 cm
`selectivity`	opal curve per fishery, and which parameters to estimate	Double-normal for fisheries, logistic for the index; estimate what SS3 estimated

```
# Every choice that cannot be read from the SS3 files. The converter reads
# nothing else, so this list is the full record of the conversion decisions.
cfg = list(
    model       = "io-yft-sim-iattc-sample-tree-4",
    # SS3 model folder, relative to dir_root in ss3_to_opal() (or set
    # absolute_paths = TRUE and give a full path).
    ss3_dir     = "model-files/ss3/xx-io-yft-sim-iattc-sample-tree-4",
    ss3_version = "3.24",
    files = list(
        dat         = "data.ss_new",
        ctl         = "control.ss_new",
        par         = "ss3.par",
        read_report = TRUE        # Report.sso: endgrowth, ageselex, SSB0, bias ramp
    ),

    # Time: 256 SS3 "years" that are quarters (nseas = 1, months_per_seas = 3).
    # opal runs the same 256 steps with n_season = 1.
    time = list(first_ts = 1, last_ts = 256),

    # Ages: drop SS3 age 0 so that the opal age index equals the SS3 age.
    age = list(keep_ss3_ages = c(1, 28), min_age = 1),

    # Growth: SS3 GrowthModel 3 (age-specific K) reproduced exactly in opal.
    # "schnute" would instead fit opal's single-k curve to SS3's length-at-age.
    growth = list(
        model             = "ss3_age_k",
        plus_group_adjust = TRUE,   # SS3's plus-group mean length
        A1 = 1, A2 = 28, fit_cv = TRUE   # used only by model = "schnute"
    ),

    # Biology: M, weight, maturity and Mat*Fecund at age straight from SS3's
    # endgrowth table ("control" would rebuild them from the control file).
    bio = list(
        source = "endgrowth",
        endgrowth = list(
            length_basis       = "Beg",
            weight             = TRUE,
            maturity           = TRUE,
            spawning_potential = TRUE,
            apply_frac_female  = FALSE,   # single-sex model: SSB has no female fraction
            m                  = TRUE
        ),
        m_rate_divisor = NULL   # NULL: 12 / (nseas * months_per_seas) = 4 here
```

```
    ),

    # Index: SS3 fleet 17 (longline CPUE) split into four quarterly series,
    # opal fisheries 17-20, each with its own catchability.
    index = list(
        keep_ss3_fleets = 17,
        split = list(list(fleet = 17, into = 17:20, by = "ts_modulo",
                          modulo = 4, offset = 81)),
        normalise            = "mean1",
        month_labels         = c(2, 5, 8, 11),
        keep_negative_fleets = FALSE
    ),

    # Length compositions: fleets with length data in SS3; the last SS3 bin
    # (195-198 cm) is regularised onto the 5 cm grid.
    lf = list(
        keep_negative_fleets = FALSE,
        regularise_last_bin  = TRUE,
        keep_fisheries       = c(1:14, 16),
        cap                  = NULL,
        addtocomp            = 1e-10
    ),

    # Selectivity. opal codes (not SS3 patterns): 1 logistic, 2 double-normal,
    # 3 double Richards. Fisheries 1-16 double-normal, index 17-20 logistic.
    selectivity = list(
        report_factor  = "Asel2",   # Report.sso ageselex rows to read
        reference_year = NULL,      # NULL: terminal year
        n_par          = 6,
        opal_type_f    = c(rep(2, 16), rep(1, 4)),
        # Estimate the opal equivalents of the parameters SS3 estimated.
        estimate = c(1:14, 17),
        mirror   = list(`15` = 14, `16` = 14, `18` = 17, `19` = 17, `20` = 17),
        free_par = list(`1` = 1:2, `2` = 1:4),
        # Parameters the length data do not inform (they settled on the prior
        # mean when estimated), held at their seeded values instead.
        fix_prior_dominated = list(`4` = c(2, 4), `5` = c(2, 4), `7` = c(2, 4),
                                   `9` = 4, `13` = c(2, 4))
    ),

    # SS3's generalised size compositions are not converted.
    sizefreq = list(include = FALSE),

    metadata = list(species = "Yellowfin tuna", region = "Indian Ocean",
                    data_type = "simulated")
)
# Provenance stored on the opal object.
cfg$config_path = "code/rtmb/scripts/05-io-yft-ss3-to-opal-workflow.r"
cfg$config_md5  = unname(tools::md5sum(file.path(proj_dir, cfg$config_path)))
```

Points that are easy to get wrong:

- **Time units.** opal's time step is the model step. When the SS3 year is a quarter, SS3 still reads M and the von Bertalanffy K as *annual* rates and applies them over the season. The converter divides both by `12 / (nseas × months_per_seas)`, which is 4 here (`bio$m_rate_divisor`).
- **Biology route.** `endgrowth` takes biology at age from SS3 exactly. Use `control` only if growth is to be estimated in opal: it rebuilds weight and maturity on a length basis and converts them through opal's length-at-age, so any difference in growth then reaches spawning biomass.
- **Selectivity codes.** `opal_type_f` uses opal's codes (1 logistic, 2 double-normal, 3 double Richards), not SS3 pattern numbers. `validate_opal_inputs()` checks this.
- **Index splitting** is a modelling choice: one SS3 index with one catchability, against four opal series with four. Leave `split` empty to keep SS3's index as one series.
- **`estimate`, `mirror`, `free_par` and `fix_prior_dominated`** are used only when the final model is set up (section 7), not during conversion.

4 Read the SS3 model

`ss3_to_opal()` reads the files itself. Reading them separately first lets you look at the model and record the features that decide how it converts.

```
# ss3_to_opal() reads the files itself; reading them here as well is for
# looking at the model before converting it.
ss3 = ss3_read_model(dir_ss3     = file.path(proj_dir, cfg$ss3_dir),
                     dat_file    = cfg$files$dat,
                     ctl_file    = cfg$files$ctl,
                     par_file    = cfg$files$par,
                     version     = cfg$ss3_version,
                     read_report = TRUE)
names(ss3)   # dat, ctl, par, rep (r4ss::SS_output) and provenance
```

```
# The SS3 features the converter depends on. Run this on your own model and
# compare it with the yellowfin column in the guide.
ss3_model_features = function(ss3) {
    dat = ss3$dat; ctl = ss3$ctl; rep = ss3$rep
    get = function(x, nm, default = NA) {
        hit = intersect(nm, names(x))
        if (length(hit)) x[[hit[1]]] else default
    }
    n_rows = function(x) if (is.null(x) || !length(x)) 0L else NROW(x)
    patterns = function(x) {
        if (is.null(x) || !NROW(x)) return("none")
        p = as.data.frame(x)[[1]]
        paste(sort(unique(p)), collapse = ", ")
    }
    yn = function(x) if (isTRUE(x)) "yes" else "no"
    data.table::data.table(
        feature = c("SS3 version", "Model period (SS3 years)", "Seasons per year",
                    "Months per season", "Sexes", "Areas", "Ages (max)",
                    "Fishing fleets", "Survey / index fleets",
                    "Catch units (1 biomass, 2 numbers)",
                    "Index observations", "Length composition rows",
                    "Age composition rows", "Generalised size comp methods",
                    "Tag release groups", "Growth model (1 vB, 2 Richards, 3 age-K)",
                    "Growth patterns", "Platoons per growth pattern",
```

```
                    "Length selectivity patterns", "Age selectivity patterns",
                    "Time-block designs", "F method", "Stock-recruit function",
                    "Report.sso available"),
        value = c(
            as.character(get(rep, "SS_version", get(dat, "ReadVersion", NA))),
            paste(get(dat, "styr"), get(dat, "endyr"), sep = "-"),
            get(dat, "nseas"),
              paste(unique(get(dat, c("months_per_seas", "month_per_seas"))), collapse =
", "),
            get(dat, "Nsexes"),
            get(dat, "N_areas"),
            get(dat, "Nages"),
            get(dat, c("Nfleet", "Nfleets")),
            get(dat, "Nsurveys", 0),
            paste(unique(get(dat, "units_of_catch", NA)), collapse = ", "),
            n_rows(get(dat, "CPUE", NULL)),
            n_rows(get(dat, "lencomp", NULL)),
            n_rows(get(dat, "agecomp", NULL)),
            get(dat, "N_sizefreq_methods", 0),
            get(dat, "N_tag_groups", 0),
            get(ctl, "GrowthModel"),
            get(ctl, "N_GP"),
            get(ctl, "N_platoon", 1),
            patterns(get(ctl, "size_selex_types", NULL)),
            patterns(get(ctl, "age_selex_types", NULL)),
            get(ctl, "N_Block_Designs", 0),
            get(ctl, "F_Method"),
            get(ctl, "SR_function"),
            yn(!is.null(rep))
        )
    )
}
features = ss3_model_features(ss3)
features
```

`ss3_model_features()` is defined in the script. Its output for IO YFT is in section 9, next to a blank column for your own model.

5 Convert

```
converted = ss3_to_opal(cfg, dir_root = proj_dir)
names(converted)   # data, parameters, lf, selectivity, meta, ss3

# Length compositions go into the data list through opal's prep_lf_data().
# SS3's mult_by_lencomp_N becomes opal's lf_var_adjust (its reciprocal).
lf_wide = dcast(converted$lf, fishery + year + month + ts ~ bin,
                value.var = "value", fill = 0) %>%
          setorder(fishery, ts) %>%
          as.data.frame()                # opal objects cannot hold data.tables
converted$data = prep_lf_data(
    converted$data, lf_wide,
    lf_switch         = 1L,
```

```
    lf_keep_fisheries = as.integer(cfg$lf$keep_fisheries),
    lf_var_adjust     = converted$meta$lf_var_adjust,
    lf_cap            = cfg$lf$cap,
    lf_addtocomp      = cfg$lf$addtocomp
) %>% ss3_make_portable()
```

`ss3_to_opal()` returns:

- `data`: the opal data list;
- `parameters`: starting values at SS3's estimates;
- `lf`: length compositions in long format;
- `selectivity`: SS3's selectivity-at-age, and the same projected onto length bins;
- `meta`: diagnostics and provenance;
- `ss3`: the r4ss objects it read.

The starting values are SS3's own estimates:

- `log_B0` is SS3's unfished spawning biomass (opal estimates B0, not R0).
- `rdev_y` holds SS3's recruitment deviations.
- Steepness and σ_R come from the par file.
- Growth comes from the SS3 files.
- Each fishery's initial F is set to effectively zero (`log_init_F_f`), so the model starts unfished.
- Catchabilities start at 1.

Then `prep_lf_data()` (an opal function) adds the length compositions to the data list. SS3's `mult_by_lencomp_N` becomes opal's `lf_var_adjust`; opal divides the sample size by it, so the converter passes the reciprocal.

Two opal rules affect anything you add to the data list:

- **Only plain R objects.** `opal_obj()` rejects data elements that hold an external pointer, and every data.table has one. Convert to `data.frame` (as above) or run `ss3_make_portable()` on the list.
- **Change an opal object only through `opal_update()`.** The object stores a hash of its contents. Assigning directly (`x$data$foo <- ...`) makes the next validation fail with "Model state was modified directly". The `ss3_use_*()` helpers all go through `opal_update()`.

6 Test the conversion

The tests run in order, from the cheapest and most specific to the most general. A failure at one level should be fixed before moving on, because the later tests cannot separate a conversion error from a real difference between the two models.

Test	Asks	Fails when
1. Structure	Is the opal input internally consistent?	dimension mismatches, wrong codes, non-portable objects
2. Reconciliation	Did the data and biology survive the conversion?	lost or duplicated observations, wrong units, wrong female fraction
3. Build	Can opal evaluate the model at the starting values?	non-finite objective or gradient
4. Bridging	At SS3's parameters, does opal reproduce SS3's dynamics?	any remaining conversion error that affects the population

6.1 Test 1: structure

```
# Test 1: is the opal input internally consistent?
checks = validate_opal_inputs(converted, strict = FALSE)
checks[, .(n_checks = .N, passed = sum(pass))]
checks[pass == FALSE]
```

`validate_opal_inputs()` checks the following:

- dimensions (ages, years, fisheries, length bins);
- that M, weight, maturity and spawning potential have a valid length and range;
- catch array shape and units codes;
- index columns, index numbering and time window;
- length composition bins and fisheries;
- parameter vector lengths;
- selectivity codes;
- that nothing non-portable is in the data list.

For IO YFT, **38 of 38** checks pass.

Check	Pass
data list is portable for opal_obj()	yes
n_age matches age_a	yes
years length matches n_year	yes
M length is n_age or n_len	yes
maturity length is n_age or n_len	yes
fecundity length is n_age or n_len	yes
maturity is a probability	yes
M is positive and finite	yes
weight length is n_age or n_len	yes
weight is positive, finite and increasing with age	yes
spawning_potential length is n_age or n_len	yes
spawning_potential is non-negative and finite	yes
spawning_potential is non-zero somewhere	yes
endgrowth route supplies M on an age basis	yes
endgrowth route supplies maturity on an age basis	yes
catch array dimensions	yes
catch is non-negative and finite	yes
catch_units_f length	yes
catch_units_f codes are 1 or 2	yes
sel_type_f length	yes
sel_type_f uses opal codes (1-3), not SS3 patterns	yes
cpue_data has required columns	yes
cpue index runs 1:n_index	yes

Check	Pass
cpue ts within model window	yes
cpue values positive and finite	yes
cpue se positive and finite	yes
lf bins match the model length structure	yes
lf ts within model window	yes
lf fisheries within 1:n_fishery	yes
lf values non-negative and finite	yes
rdev_y length matches n_year	yes
init_rdev_a length matches n_age	yes
log_cpue_q length matches n_index	yes
par_sel rows match n_fishery	yes
growth parameters are finite	yes
L2 exceeds L1	yes
sel_fa_external dimensions	yes
sel_fa_external within [0, 1]	yes

6.2 Test 2: reconciliation with SS3

```
# Test 2: did the data and biology survive the conversion?
recon = ss3_reconcile(converted, tol = 1e-6)
recon[, .(quantity, ss3, opal, rel_diff, pass)]
```

`ss3_reconcile()` compares quantities that should match SS3 exactly. Each has a relative tolerance of 10^{-6}, except spawning potential per recruit, which has 10^{-4}. The most informative row is **unfished spawning potential per recruit**: one number that would show M in the wrong time units (a factor of about 4 here), a misplaced female fraction (a factor of 2) or a wrong plus-group treatment. Its small residual comes from two places: Report.sso prints six significant figures, and SS3 3.24 closes the plus group differently from opal.

Quantity	SS3	opal	Relative difference	Pass
catch total, fleet 1 (fishing_gi_1)	58644.84	58644.84	0.0e+00	yes
catch total, fleet 2 (fishing_gi_4)	19412.76	19412.76	0.0e+00	yes
catch total, fleet 3 (fishing_hd_1)	47115.22	47115.22	0.0e+00	yes
catch total, fleet 4 (fishing_ll_1)	47074.37	47074.37	0.0e+00	yes
catch total, fleet 5 (fishing_ll_2)	9146.365	9146.365	0.0e+00	yes
catch total, fleet 6 (fishing_ll_3)	4744.187	4744.187	0.0e+00	yes
catch total, fleet 7 (fishing_ll_4)	33189.32	33189.32	0.0e+00	yes
catch total, fleet 8 (fishing_other_1)	58433.02	58433.02	0.0e+00	yes
catch total, fleet 9 (fishing_other_4)	40323.64	40323.64	0.0e+00	yes
catch total, fleet 10 (fishing_bb_1)	138227.8	138227.8	0.0e+00	yes
catch total, fleet 11 (fishing_ps_1)	369226.9	369226.9	0.0e+00	yes

Quantity	SS3	opal	Relative difference	Pass
catch total, fleet 12 (fishing_ps_2)	30289.96	30289.96	0.0e+00	yes
catch total, fleet 13 (fishing_ps_4)	9416.365	9416.365	0.0e+00	yes
catch total, fleet 14 (fishing_trol_1)	37630.63	37630.63	0.0e+00	yes
catch total, fleet 15 (fishing_trol_2)	32907.52	32907.52	0.0e+00	yes
catch total, fleet 16 (fishing_trol_4)	92779.85	92779.85	0.0e+00	yes
CPUE observation count	176	176	0.0e+00	yes
LF total input sample size	5210	5210	0.0e+00	yes
LF observation rows	1114	1114	0.0e+00	yes
unfished spawning potential per recruit	39.99116	39.99303	4.7e-05	yes
total weight summed over ages	939.4881	939.4881	0.0e+00	yes
mean length-at-age, SS3 age-specific K (max relative error)	—	—	4.6e-06	yes

The length-at-age row compares opal's growth (`get_growth()` with SS3's parameters and K multipliers) with SS3's `Len_Beg`. Its error is at the level of Report.sso's print precision, so the growth conversion is exact. With `growth$model = "schnute"` this row would report how well a single-k curve approximates SS3's growth.

6.3 Test 3: build the opal object

```
# The opal assessment object. ss3_opal_obj() adds opal's default priors and
# the SS3 provenance, and opal_build() checks that the objective and gradient
# are finite at the starting values (the converted SS3 optimum).
assessment = ss3_opal_obj(converted, cfg,
                          control = list(eval.max = 10000, iter.max = 10000),
                          build   = TRUE)
summary(assessment)
```

`ss3_opal_obj()` does three things:

- attaches opal's default priors (`get_priors()`);
- records the SS3 file checksums, r4ss version and settings in `$provenance$metadata`;
- runs `opal_build()`, which builds the RTMB tape and checks that the objective and gradient are finite at the starting values.

The default map (`get_map()`) estimates `log_B0`, the recruitment deviations and the catchabilities, and fixes growth and selectivity.

6.4 Test 4: bridging at SS3's parameters

This is the main test of the conversion. SS3's selectivity-at-age is put into opal directly (`sel_fa_external`), every parameter is left at SS3's value, and nothing is estimated. opal's spawning biomass is then compared with SS3's.

```
# Test 3: with SS3's own selectivity-at-age and parameters, does opal
# reproduce SS3's spawning biomass? Nothing is estimated here.
bridge_obj = ss3_use_external_selectivity(
    assessment, converted$selectivity$sel_fa_external) %>%
    opal_build()
bridge = ss3_bridge_table(converted, opal_report(bridge_obj))
```

```
# ss3: SS3 SSB. opal: opal at SS3's values. replica_ss3: the same converted
# inputs replayed with SS3's catch and recruitment timing.
bridge[, .(series   = c("opal", "replay with SS3 timing"),
           max_abs  = c(max(abs(rel_opal)), max(abs(rel_replica_ss3))),
           mean_abs = c(mean(abs(rel_opal)), mean(abs(rel_replica_ss3))))]
```

Even with every input converted exactly, the two models differ in two places:

- **Catch timing.** opal removes each step's catch at the start of the step, before natural mortality. SS3 applies fishing and natural mortality together through the Baranov equation.
- **Recruitment timing.** opal recruits the first age class at $t + 1$ from spawning biomass at $t + 1$. With SS3 age 0 dropped, SS3's matching fish were recruited from spawning biomass at t.

`ss3_bridge_replica()` replays the population on the converted inputs, either with opal's timing or with SS3's:

- With opal's timing it reproduces `opal_report()`.
- With SS3's timing it should reproduce SS3's spawning biomass.

A small residual under SS3's timing shows that the conversion is correct. Any gap left between opal and SS3 is then a real difference between the platforms, not a conversion error.

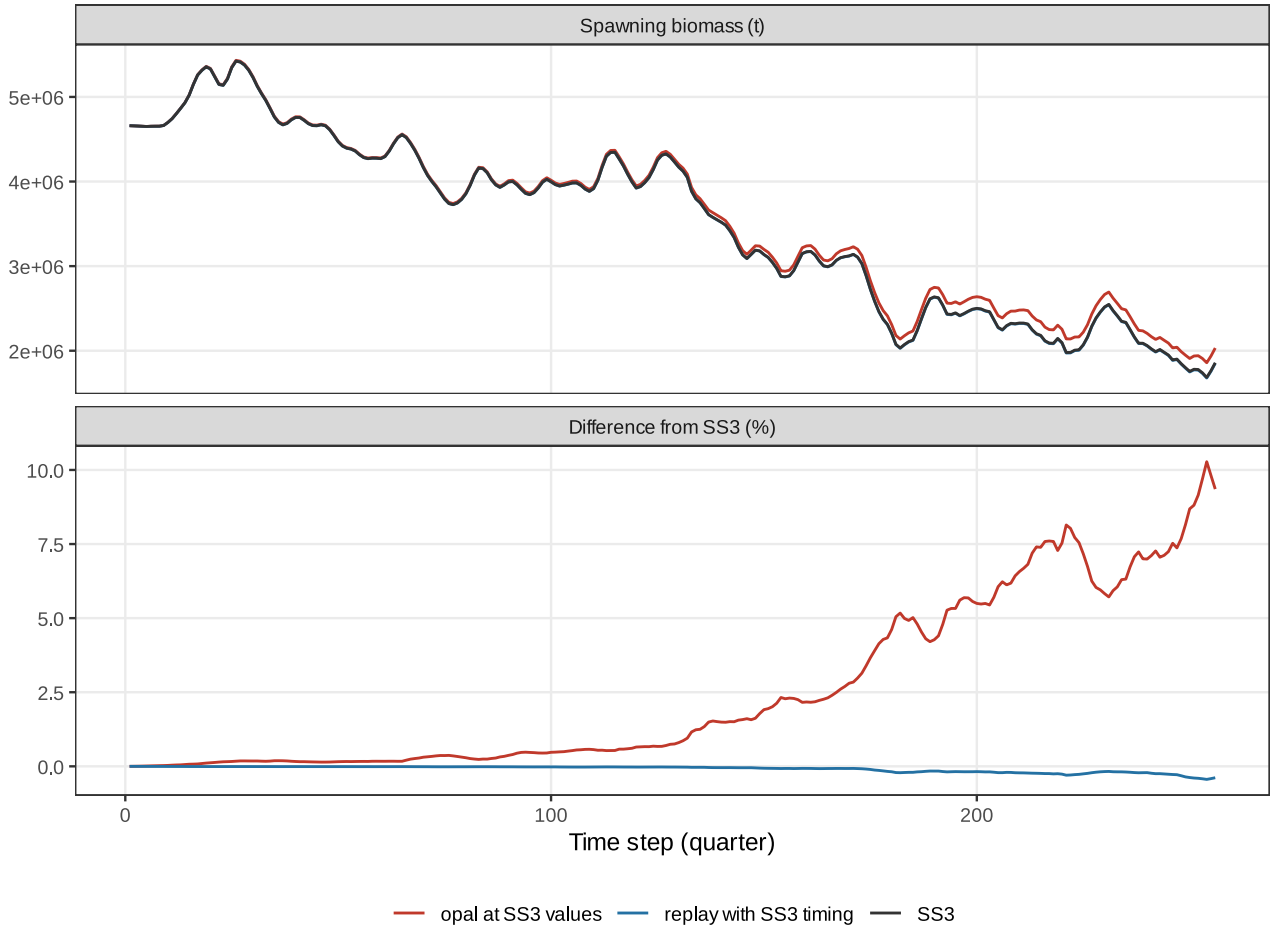


Figure 1: Bridging test. Top: spawning biomass from SS3, from opal at SS3's parameter values with SS3's selectivity-at-age, and from the converted inputs replayed with SS3's catch and recruitment timing. Bottom: each relative to SS3.

Series	Mean |difference| from SS3	Max |difference| from SS3
opal at SS3 values	2.54%	10.28%
converted inputs, replayed with SS3 timing	0.09%	0.44%

For IO YFT, opal at SS3′s values differs from SS3 by up to 10.3%. The replay with SS3′s timing differs by at most 0.44%. The conversion is therefore correct, and the opal–SS3 gap at fixed parameters comes from catch timing. The simulations in `code/rtmb/sims/` measure how much that timing difference matters in estimation.

If the replay with SS3′s timing does *not* match SS3, look for a conversion error. Common causes:

- catch units (biomass vs numbers) or a fleet out of order;
- M or K not rescaled to the time step;
- a female fraction applied twice or not at all;
- selectivity read for the wrong fleet, year or `Factor` (`Asel` vs `Asel2`);
- recruitment deviations misaligned by a step.

The replay needs biology at age and SS3′s selectivity, so it works only with the `endgrowth` route.

6.5 Starting objective and gradient

```
# Test 4: the starting objective and gradient. A non-finite value or a very
# large gradient points at a parameter or data element on the wrong scale.
start_check = function(x) {
    obj = opal_rtmb(x, fresh = TRUE)
    data.table(n_par = length(obj$par), objective = obj$fn(obj$par),
               max_abs_gradient = max(abs(obj$gr(obj$par))))
}
start_check(bridge_obj)
```

Configuration	Estimated parameters	Objective	Max |gradient|
bridge (SS3 selectivity)	261	14,242.066	2.72e+02
final (C2)	310	14,920.569	1.43e+03

The objective must be finite. At the converted SS3 optimum, the gradient is not zero, because the two models' likelihoods differ, but it should not be huge. A very large gradient on one parameter usually means a scale error in that parameter or in the data it fits.

7 Set up and fit the final model

The bridging configuration used SS3′s selectivity as fixed data. The final model, C2, uses opal's own length-based selectivity curves instead, with the opal equivalent of each parameter SS3 estimated being estimated here too.

7.1 Seed opal's selectivity from SS3′s

SS3′s selectivity here is on age, while opal's is on length, and there is no algebraic conversion between the two. `ss3_seed_par_sel()` projects SS3′s selectivity-at-age onto the length bins using SS3′s age–length key. It then fits each opal curve to that projection by least squares, using opal's own selectivity functions so that the seeded values are on opal's scale.

```
# SS3's selectivity is on age, opal's on length. Starting values come from
# fitting each opal curve to SS3's selectivity projected onto length bins.
```

```
seed = ss3_seed_par_sel(
    sel_target_fl = converted$selectivity$sel_target_fl,
    len_mid       = converted$data$len_mid,
    sel_type_f    = converted$data$sel_type_f,
    n_par         = cfg$selectivity$n_par)
data.table(fishery = seq_along(seed$rmse), rmse = round(seed$rmse, 4))
```

Fishery	opal curve	RMSE against SS3
1: fishing_gi_1	double-normal	0.026
2: fishing_gi_4	double-normal	0.021
3: fishing_hd_1	double-normal	0.020
4: fishing_ll_1	double-normal	0.014
5: fishing_ll_2	double-normal	0.008
6: fishing_ll_3	double-normal	0.012
7: fishing_ll_4	double-normal	0.015
8: fishing_other_1	double-normal	0.012
9: fishing_other_4	double-normal	0.011
10: fishing_bb_1	double-normal	0.050
11: fishing_ps_1	double-normal	0.007
12: fishing_ps_2	double-normal	0.014
13: fishing_ps_4	double-normal	0.007
14: fishing_trol_1	double-normal	0.015
15: fishing_trol_2	double-normal	0.015
16: fishing_trol_4	double-normal	0.015
17: llcpue_q1	logistic	0.014
18: llcpue_q2	logistic	0.014
19: llcpue_q3	logistic	0.014
20: llcpue_q4	logistic	0.014

The RMSE is on the selectivity scale (0–1) and shows how well the chosen opal curve can follow SS3's shape. A large value for a fishery means that curve family does not fit SS3's shape, so change `opal_type_f` for that fishery before fitting.

7.2 Choose what to estimate

`ss3_use_estimated_selectivity()` removes SS3's selectivity from the model, puts in the seeded curves and writes the selectivity part of the map from `cfg$selectivity`:

- **`estimate`**: fisheries whose curves are estimated. IO YFT: the 14 fishing fleets with their own selectivity in SS3, and the index.
- **`free_par`**: which curve parameters are estimated, by opal curve family. For the double-normal, columns 1–4 (peak, top width, ascending and descending widths) are estimated, as SS3 estimated P1–P4 of its pattern 20. For the logistic, both parameters are estimated.
- **`mirror`**: fisheries sharing another fishery's parameters (SS3 pattern 15). Fisheries 15 and 16 mirror 14; the four quarterly index series share one curve.

- **`fix`**: parameters held at their seeded values. In an earlier fit (C), these nine went to the mean of opal's selectivity prior, which meant the length data did not inform them, so C2 fixes them.

```
# Fit C2: opal's own length-based selectivity, estimated where SS3 estimated
# it, mirrored where SS3 mirrored it, with the prior-dominated parameters
# fixed. log_B0, the recruitment deviations and the four catchabilities are
# estimated as in opal's default map; growth stays at SS3's values.
final = assessment %>%
    ss3_use_estimated_selectivity(seed$par_sel, cfg,
                                  fix = cfg$selectivity$fix_prior_dominated) %>%
    opal_build()
start_check(final)

# What is estimated: map entries that are not NA.
estimated = rbindlist(lapply(names(final$parameters), function(p) {
    m = final$map[[p]]
    n = if (is.null(m)) length(final$parameters[[p]]) else length(unique(na.omit(m)))
    data.table(parameter = p, n_estimated = n)
}))[n_estimated > 0]
estimated
```

Parameter	Estimated
log_B0	1
log_cpue_q	4
rdev_y	256
par_sel	49

The 310 estimated parameters are `log_B0`, the 256 recruitment deviations, the four catchabilities and 49 selectivity parameters. Growth, steepness, σ_R and the observation variances stay at their SS3 or default values (opal's default map).

7.3 Fit

```
# Two nlminb passes, then Newton steps to bring the gradient down, then
# opal_check() on the polished optimum.
t0  = proc.time()[["elapsed"]]
fit = opal_fit(final, n_passes = 2L, check = FALSE)
obj = opal_rtmb(fit, fresh = TRUE)
pol = ss3_newton_polish(obj, fit$fit$opt$par)
opt = fit$fit$opt
opt$par       = setNames(pol$par, names(opt$par))
opt$objective = pol$objective
fit = opal_attach_fit(fit, opt, check = TRUE,
                      check_args = list(gradient_tolerance = 1e-4))
fit_seconds = proc.time()[["elapsed"]] - t0

convergence = data.table(
    objective_start = start_check(final)$objective,
    objective_fit   = opt$objective,
    max_gradient    = pol$max_gradient,
    newton_steps    = pol$steps,
```

```
    check_passes    = isTRUE(fit$validation$fit$passes),
    positive_hessian = isTRUE(fit$validation$fit$metrics$positive_hessian),
    seconds         = fit_seconds)
convergence
```

`opal_fit()` runs `nlminb` (here two passes). `nlminb` stops on relative change in the objective, which for an objective of this size can leave gradients near 10^{-3}. `ss3_newton_polish()` then takes a few Newton steps with the exact RTMB Hessian. `opal_attach_fit()` puts the polished optimum back on the object and runs `opal_check()`, which checks the gradient (here to 10^{-4}), that the Hessian is positive definite and that the parameters are estimable.

Objective at start	Objective at fit	Max |gradient|	Newton steps	Positive definite Hessian	opal_check()	Seconds
14,920.57	14,406.585	8.0e-12	2	yes	passed	103

7.4 Save

```
fit_path = file.path(dir_out, "io-yft-workflow-fit.rds")
opal_save(fit, fit_path, compress = "xz", overwrite = TRUE)
# Later: fit = opal_read(fit_path, rebuild = TRUE)
```

`opal_save()` stores the whole object: data, parameters, map, fit and provenance. `opal_read(path, rebuild = TRUE)` loads it again and rebuilds the RTMB tape, ready for `opal_report()`.

8 A first look at the fit

These figures only check that the fitted model is sensible and close to SS3. The two vignettes listed in section 1 have the full comparison, including against the simulated truth. The companion script's `results` chunk builds these tables from `opal_report(fit)` and SS3's `Report.sso`, and can be reused as it is.

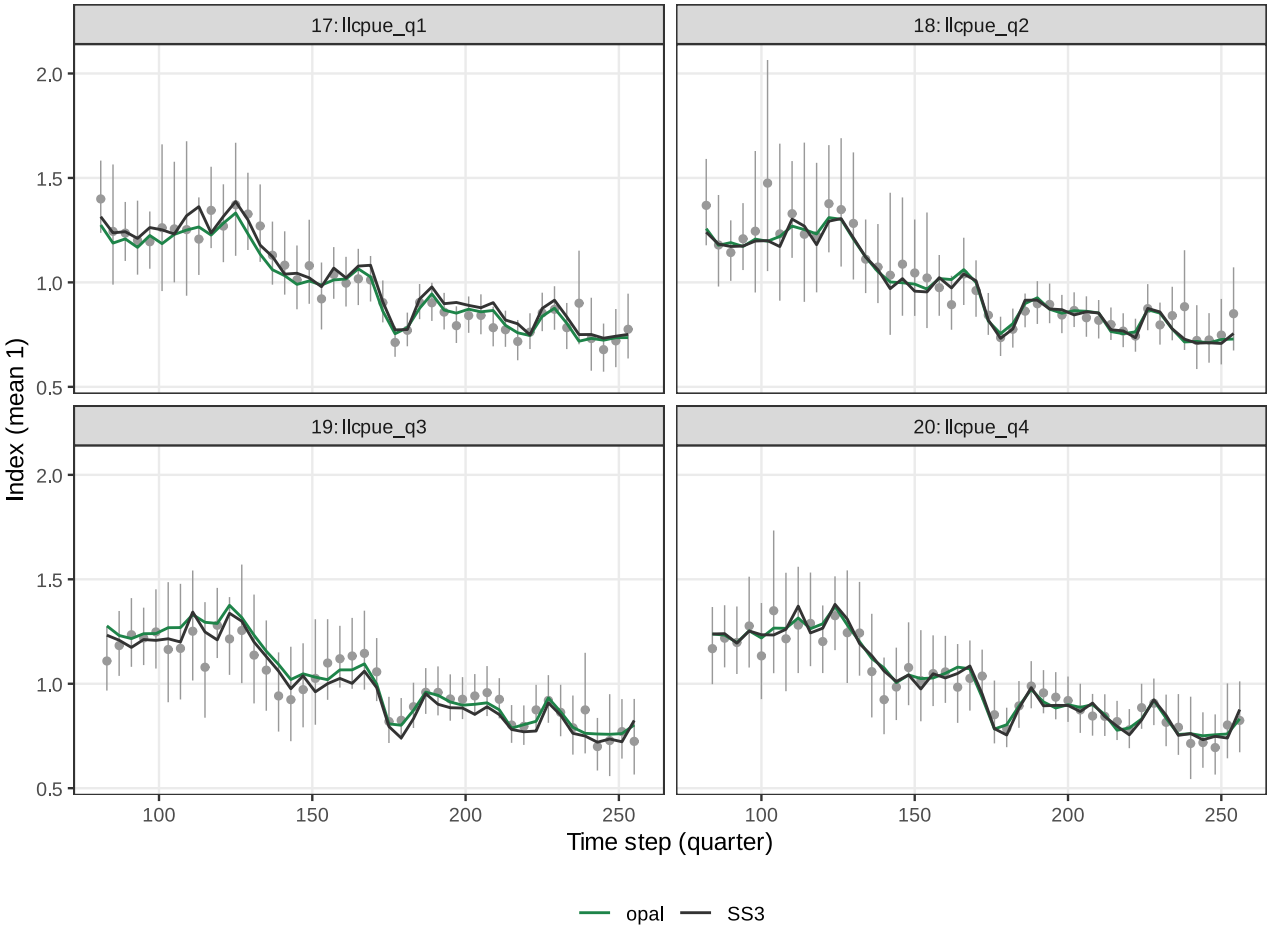


Figure 2: Longline index, each quarterly series scaled to mean 1: observations (points, ±1.96 SE), opal's fit and SS3's expected values.

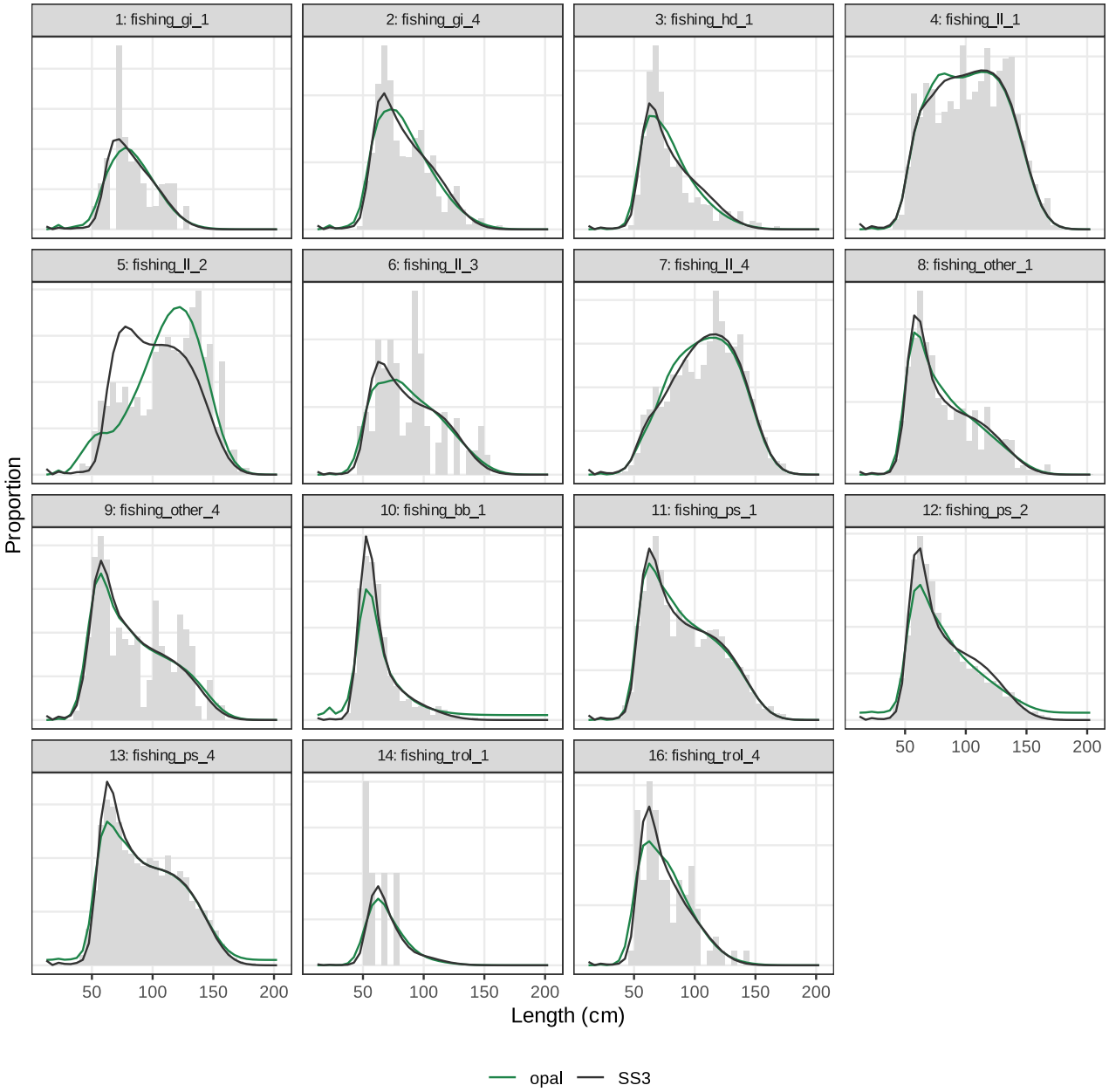


Figure 3: Length compositions aggregated over all observations for each fishery, weighted by sample size: observed (bars), opal's and SS3's expected.

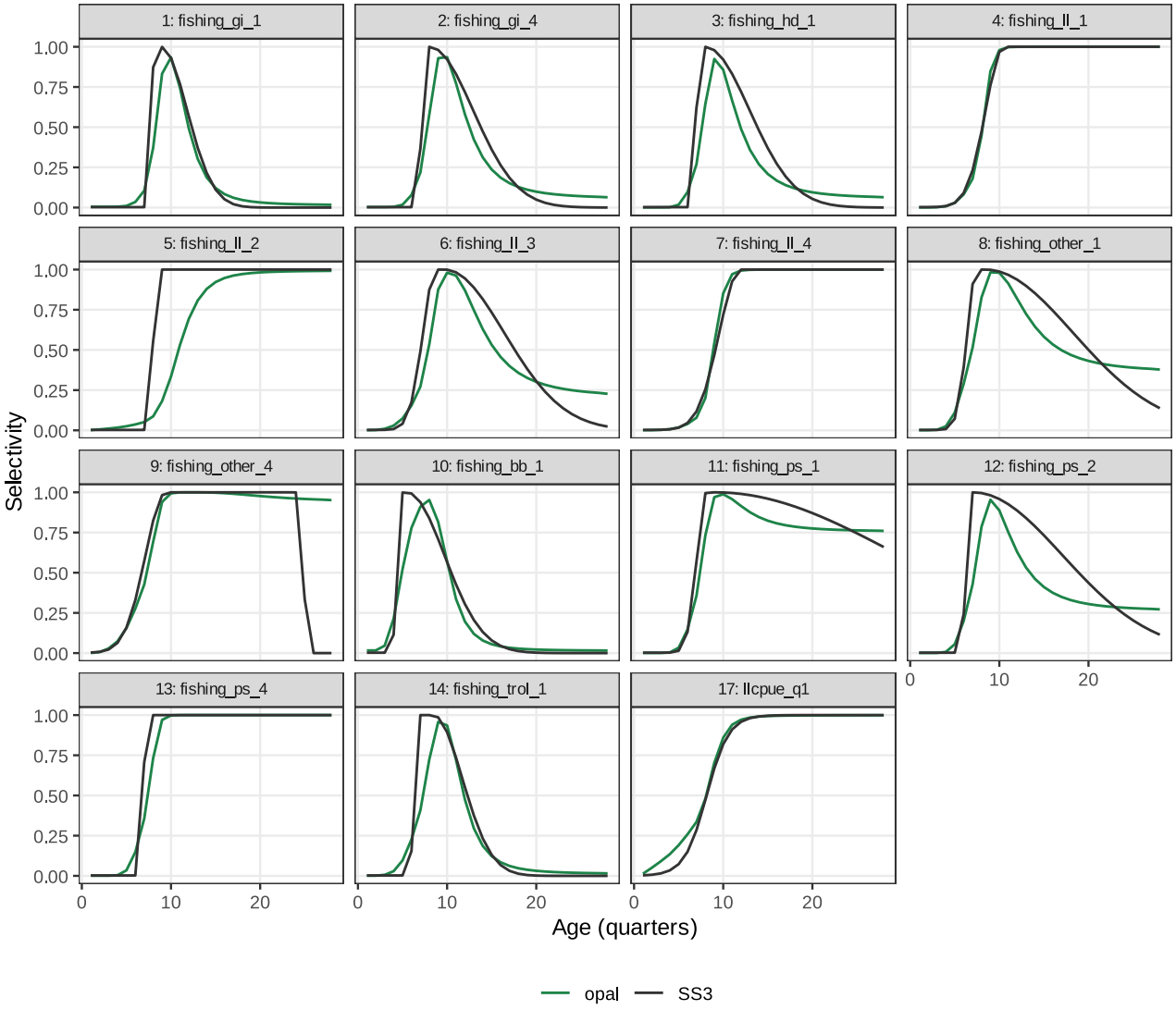


Figure 4: Selectivity-at-age: SS3's (age-based) and opal's fitted length-based curves mapped to age through the age–length key. Fisheries 15, 16 and 18–20 are mirrors and are not shown.

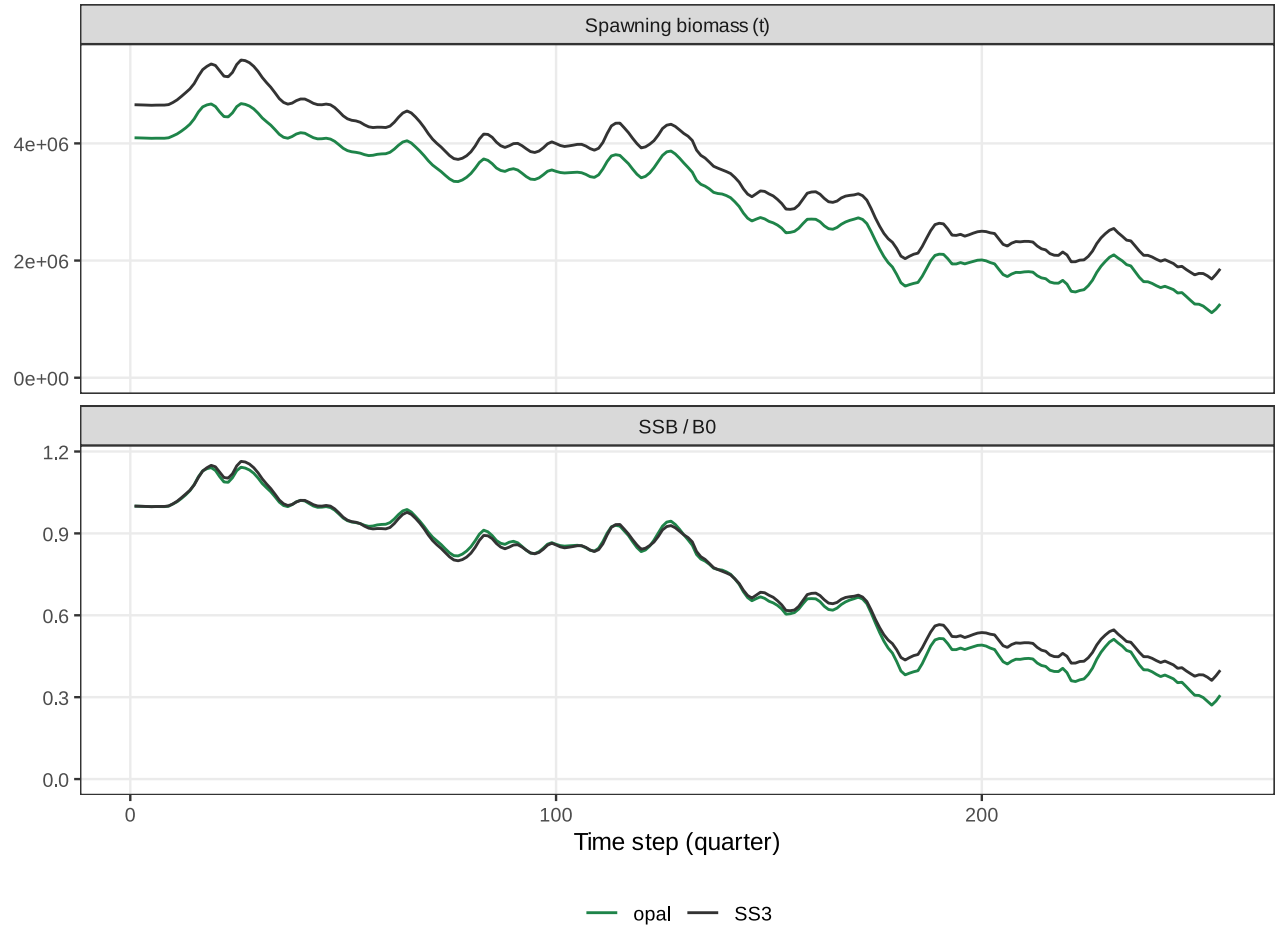


Figure 5: Spawning biomass (top) and spawning biomass relative to B0 (bottom): SS3 and the fitted opal model.

Quantity	SS3	opal	opal / SS3
B0 (unfished SSB)	4,659,770	4,095,647	0.88
Terminal SSB	1,858,230	1,258,097	0.68
Terminal SSB / B0	0.399	0.307	0.77

For IO YFT, opal's B0 is 0.88 × SS3's, and its terminal SSB/B0 is 0.31 against SS3's 0.40. Most of that gap comes from selectivity. When the same opal model is fitted with SS3's selectivity-at-age held fixed (fit A1 in `ss3-to-opal-io-yft-age-k.qmd`), its terminal SSB/B0 is within 0.01 of SS3's. The remaining sources of difference are:

- catch timing;
- opal's length-based selectivity curves, which cannot match every SS3 age-based shape;
- the index split into four series with separate catchabilities;
- SS3 data that opal does not use (here, the survey's generalised size compositions).

What matters is that the gap can be explained. The two vignettes explain the IO YFT gap one cause at a time.

9 From yellowfin to your model

The converter was written for, and tested on, this one SS3 model. It reads the general SS3 file layout through r4ss, but wherever IO YFT did not need a feature, the code does not handle it. Before converting another model, list its features with `ss3_model_features()` and compare them with IO YFT's. Each difference tells you which function to check or extend. The last column points to where to start.

Feature	IO YFT	Your model	What the converter does / where to look
SS3 version	3.24AB		Tested on 3.24 only. `ss3_read_model()` reads the par file with `SS_readpar_3.24()`, and `ss3_lencomp_n_mult()` and the catch row of `ss3_reconcile()` expect 3.24 layouts. For 3.30, check every reader in `ss3-convert.R`, and `ss3_recdevs()`.
Model period (SS3 years)	1-256		`cfg$time`; the opal step is the SS3 year.
Seasons per year	1		The converter assumes `nseas = 1`: opal gets `n_season = 1`, and `ss3_catch_to_opal()` keeps one catch value per year and fleet.
Months per season	3		Sets the M and K divisor (`ss3_m_rate_divisor()`).
Sexes	1		Single-sex only. `ss3_endgrowth_table()` keeps the first sex; two-sex compositions and spawning output need new code.
Areas	1		Single area only.
Ages (max)	28		`cfg$age`.
Fishing fleets	16		One opal fishery per SS3 fleet, in SS3 order.
Survey / index fleets	1		`cfg$index` (`keep_ss3_fleets`, `split`); opal adds one fishery per index series.
Catch units (1 biomass, 2 numbers)	2		Passed through (opal uses the same codes).
Index observations	176		`ss3_index_to_opal()`; negative fleet codes are dropped; scaled to mean 1.
Length composition rows	1114		`ss3_lencomp_to_opal()`; one sex, no partitions. `mult_by_lencomp_N` becomes `lf_var_adjust`.
Age composition rows	0		Not converted. Conditional age-at-length or age data need new code.
Generalised size comp methods	1		Not converted (IO YFT's survey size data are dropped).
Tag release groups	0		Not converted.
Growth model (1 vB, 2 Richards, 3 age-K)	3		3: `growth$model = "ss3_age_k"` (exact). 1: `"schnute"`, which matches von Bertalanffy. 2: Schnute approximation only; check the length-at-age row of `ss3_reconcile()`.
Growth patterns	1		One expected.
Platoons per growth pattern	1		One expected. With platoons, `ss3_endgrowth_table()` averages rows without abundance weights and warns.
Length selectivity patterns	0		Any pattern works for the bridging test (selectivity is read from `ageselex`). For the final model, choose `opal_type_f` to suit SS3's shapes and check the seeding RMSE.

Feature	IO YFT	Your model	What the converter does / where to look
Age selectivity patterns	12, 15, 20		As above.
Time-block designs	0		Only one year of selectivity is used (`selectivity$reference_year`). Time-varying selectivity is not represented.
F method	3		opal removes catch at the start of each step; the bridging test measures the effect.
Stock-recruit function	3		opal is Beverton–Holt; steepness and σR are read from the par file.
Report.sso available	yes		Required for the `endgrowth` route, selectivity, B0 and the bridging test.

The converter also assumes the following, which are not in the table:

- **Initial conditions.** The converter starts the population unfished: initial F is effectively zero and there are no initial age deviations. If your model starts from a fished equilibrium (initial equilibrium catch or initial F), `ss3_pars_to_opal()` has to set `log_init_F_f` and the map has to estimate it.
- **Recruitment deviations.** Early and main deviations are read from the par file (`ss3_recdevs()`), and SS3's bias-adjustment ramp from Report.sso. Forecast deviations are ignored.
- **Parameters with environmental links, time variation or deviations** (other than recruitment) are not converted. Their effects on selectivity and biology are captured only through the single reference year read from Report.sso.

A suggested order for a new model:

1. Run `ss3_model_features()` and fill in the "Your model" column.
2. Set up `cfg` and get `ss3_to_opal()` to run, extending the converter where the table says it needs to.
3. Make tests 1 and 2 pass exactly before anything else.
4. Run the bridging test and make the replay with SS3 timing match SS3.
5. Only then set up the opal selectivity and fit.

10 Credit

The SS3 model used here (`model-files/ss3/xx-io-yft-sim-iattc-sample-tree-4`) was developed by the IATTC analyst team for the Spatial Stock Assessment Simulation Experiment: Mark N. Maunder, Haikun Xu and Carolina Minte-Vera (Inter-American Tropical Tuna Commission). The yellowfin assessments from the experiment, including theirs, are evaluated in Goethel, D.R., et al. (2024). 'Drivin' with your eyes closed': Results from an international, blinded simulation experiment to evaluate spatial stock assessments. *Fish and Fisheries* 25(3): 471–490. https://doi.org/10.1111/faf.12819

11 Session

Item	Value
cache written	2026-10-15 01:18:24 UTC
r	4.5.3
opal	0.1.1
r4ss	1.56.0